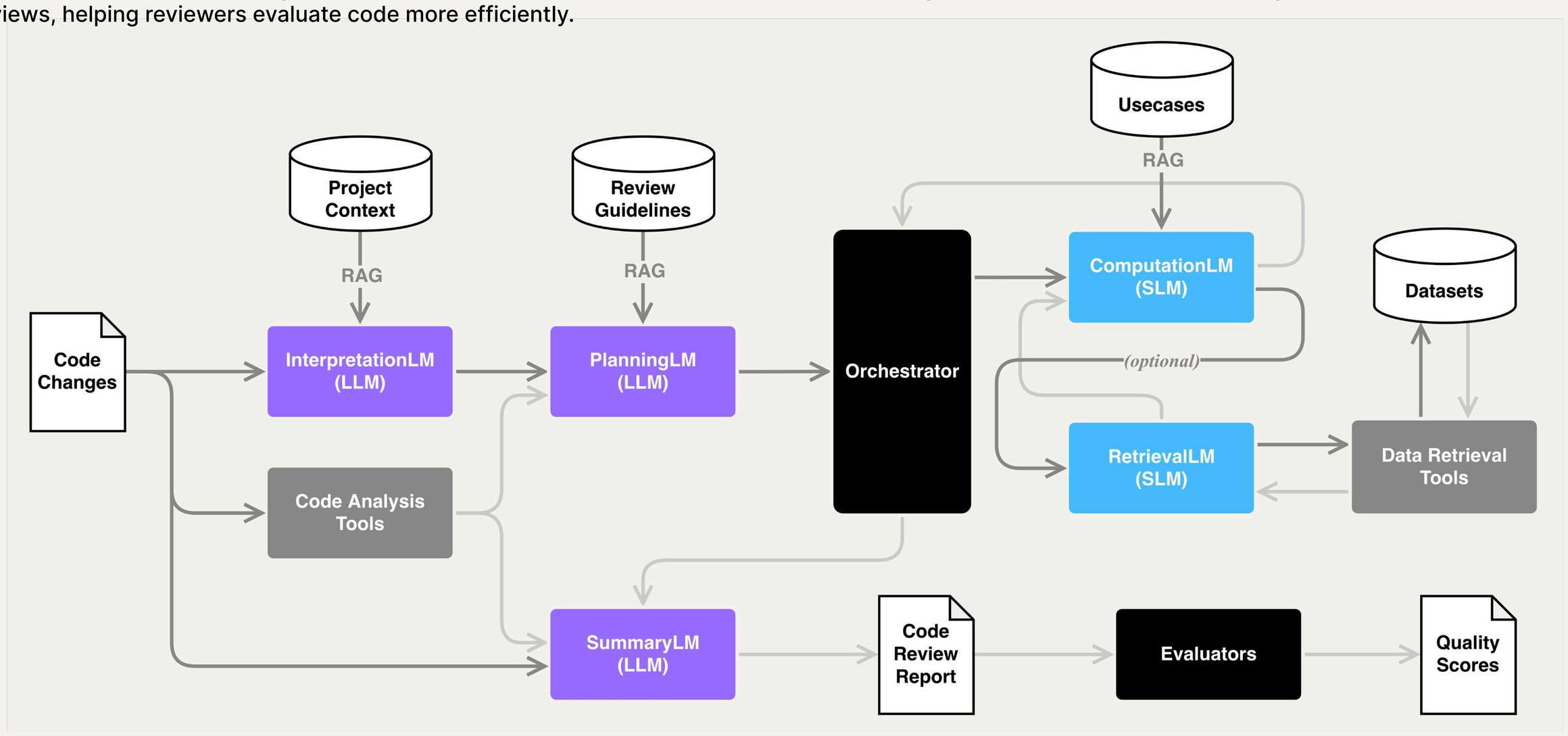
Automated Code Review Using a Multi-LLMs Framework

Po-Yun Cheng[1]* Kuan-Yi Lee[1] Chen-Wei Zhang[1] Jonathan Lee[1]
[1]Department of Computer Science and Information Engineering, National Taiwan University
***Email: b11902038@ntu.edu.tw**

Introduction

AI generates code at a faster pace than human reviewers can verify, so an automated system is important to increase reviewers' efficiency and enhance review quality. Most related works focus on a single LLM combined with different prompting, retrievalaugmented generation, or tool-calling techniques.

We propose **LLMCR, an intelligent multi-LLM code review system** that focuses on validating Java pull requests and delivering comprehensive code reviews, helping reviewers evaluate code more efficiently.



Overview of the stage-based code review system. For InterpretationLM, PlanningLM, and SummaryLM, we use Gemini-3.1-flash-lite. For ComputationLM and RetrievalLM, we use Nvidia-Nemotron-3-4B to reduce cost.

Methodology

We investigate the reasoning process of a human reviewer and summarize it into four reasoning steps:
1) Interpreting code changes, 2) Planning what to check, 3) Analyzing individual check items and 4) Summary the results into a review.

We then design our multi-LLM framework, where each stage maps to a specific human reasoning step. This design allows us to:

- Separate the information needed for different stages, avoiding putting all context into a single iteration.
- Keep the process transparent, ensuring the model follows the same reasoning process as a human.
- Allow different stages to use specialized models.

At each stage, we use retrieval-augmented generation (RAG) to provide relevant project context and knowledge to the model, enabling more informed and context-aware code review. For any project to be used, we first index its project context using an Extract, Transform, Load (ETL) pipeline, which is responsible for extracting paragraphs from documents, transforming them into semantically meaningful chunks, and loading these chunks into a vector database. WWe currently use Spring AI as our test project, using its full source code repository and documentation as the project context.

Evaluation

We evaluate on 10 real pull requests from the Spring AI GitHub repository, selected by two criteria: PR description >50 words, and >10 human comments. We compare four systems: CodeRabbit, GitHub Copilot, SingleLLM (our system with only the summary stage), and our full system.

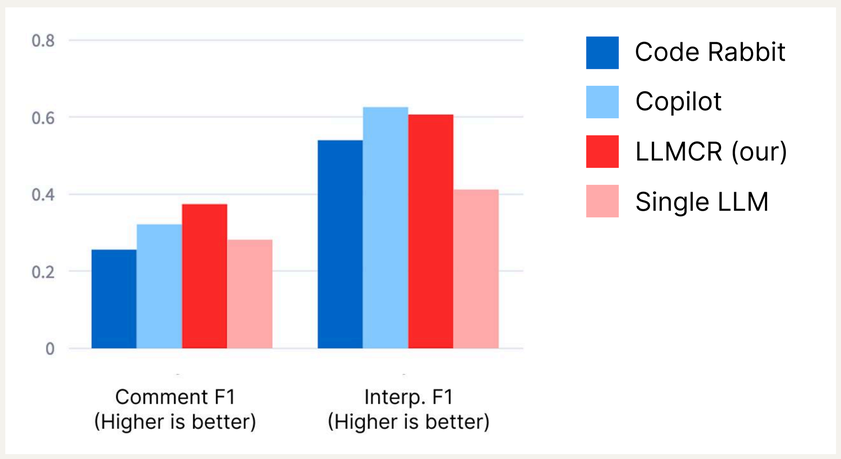
Comment-alignment: measures how closely the generated review matches human comments. Human comments are rephrased by an LLM into atomic reference sentences; generated reviews are split into candidate sentences; SentenceBERT matches candidate–reference pairs to compute Precision / Recall / F1.

Interpretation-alignment: measures how well the generated code-change interpretation matches the original PR description, computed the same way but using the PR description as the reference.

Results

Comment-alignment: LLMCR (our system) achieved the highest F1 score among all groups, showing its reviews align most closely with human comments.

Interpretation-alignment: LLMCR scored lower than Copilot but slightly higher than CodeRabbit — indicating it's not yet as strong at understanding code changes.



Groups	Review Time (per thousand lines of change)	Input and Output Tokens (per review)	Cost (per review)
LLMCR	∼10 minutes	(gemini-3.1-flash-lite) 61452 / 1568 (nemotron-3-4b) 1820242 / 4905	0.02 usd
Single LLM	<10s	(gemini-3.1-flash-lite) 14615 / 2069	0.007 usd
CodeRabbit	∼10 minutes	x	0.25 usd
GitHub Copilot	∼5 minutes	x	0.5 usd

Conclusion

Our proposed system achieves performance comparable to existing code review tools while using less powerful models and incurring lower inference costs. However, the designed system fails to effectively provide the project context required by the LLMs, due to the lack of suitable datasets.

For future work, we plan several directions to partially mitigate this data limitation: 1) collect GitHub issues to better understand current project concerns, 2) infer coding conventions from source code and existing documents, 3) infer development considerations from past review comments, and 4) manually rewrite and organize the review guidelines.